30.3. Python: SymPy

Symbolic Computation using SymPy

Previous Section:

</> 30.2. Python: SciPy

Contents:

Up to this point, we have worked primarily with **numerical computation**.

- When NumPy computes: `2 + 3`, it immediately returns `5` .
- When SciPy computes a derivative numerically: $f'(2)$, it returns a numerical approximation at a particular point.

But mathematicians often want something different. Instead of asking: What is the derivative at $x = 2$? They ask: ***What is the derivative itself?*** This is where **SymPy** becomes useful.

SymPy (Symbolic Python) is a Python library for symbolic mathematics. Instead of immediately evaluating expressions into numbers, SymPy keeps the mathematical expression itself and manipulates it symbolically.

For example:

```
import sympy as sp

x = sp.Symbol('x')
expr = x**2 + 3*x + 1

print(expr)
```

```
x**2 + 3*x + 1
```

The variable `x` is not assigned a value. It remains a mathematical symbol.

This allows SymPy to perform algebra, calculus, equation solving, and many other operations exactly rather than approximately.

3. Common Operations on SymPy

3.1. Square Roots

Normally, `math.sqrt(2)` returns `1.41421356237...` , which is only an approximation. SymPy preserves the exact value.

```
import sympy as sp

print(sp.sqrt(2))
```

```
sqrt(2)
```

We can convert it to decimal form later using `sp.N(sp.sqrt(2))`:

This exact representation becomes extremely important in symbolic mathematics.

3.2. Derivatives

One of SymPy's most famous features is symbolic differentiation.

```
import sympy as sp

x = sp.Symbol('x')
f = x**3 + 2*x**2 + x

# Differentiate
print(sp.diff(f, x))
```

3*x**2 + 4*x + 1

Notice something important. SciPy numerical differentiation gives `f'(2) = 21`. But SymPy gives: `f'(x) = 3x² + 4x + 1` , the actual derivative function.

Optionally, compute the second and third derivative:

```
print(sp.diff(f, x, 2))
print(sp.diff(f, x, 3))
```

2*(3*x + 2) → **6*x + 4**
6

3.3. Integrals

SymPy can also perform symbolic integration.

```
f = x**2

# Indefinite integral
print(sp.integrate(f, x))
```

x**3/3 which represents: $\int x^2 dx$

```
# Definite Integrals
print(sp.integrate(x**2, (x, 0, 2)))
```

Definite integrals $\int_0^2 x^2 dx \rightarrow 8/3$

Notice that SymPy returns the exact fraction rather than a decimal approximation.

3.4. Partial Derivatives

Partial derivatives are fundamental in: Machine Learning; Optimization; Physics; Engineering

```
x, y = sp.symbols('x y')
f = x**2 * y + y**2

# Partial derivative with respect to x
print(sp.diff(f, x))

# Partial derivative with respect to y
print(sp.diff(f, y))
```

2*x*y
x**2 + 2*y

This concept becomes extremely important when studying gradients and neural networks.

3.5. Limits

Limits describe what happens as a variable approaches a value.

Example: $\lim_{x \to 0} \frac{\sin(x)}{x}$

```
import sympy as sp

x = sp.Symbol('x')

expr = sp.sin(x)/x
print(sp.limit(expr, x, 0))
```

1

```
# Infinite Limits
print(sp.limit(1/x, x, sp.oo))
```

0 → sp.oo represents infinity.

3.6. Series Expansion

Many complicated functions can be approximated using polynomials. This idea leads to the famous Taylor and Maclaurin Series.

- Maclaurin Series expands e^x around $x = 0$

```
x = sp.Symbol('x')
f = x**3 + 2*x**2 + x

print("Maclaurin Series:", sp.exp(x).series(x, 0, 6))
```

Maclaurin Series: 1 + x + x**2/2 + x**3/6 + x**4/24 + x**5/120 + O(x**6)

- Taylor Series expands around $x = 1$

```
print("Taylor Series:", sp.exp(x).series(x, 1, 5))
```

Taylor Series: E + E*(x - 1) + E*(x - 1)**2/2 + E*(x - 1)**3/6 + E*(x - 1)**4/24 + O((x - 1)**5, (x, 1))

Series expansions are widely used in Physics; Engineering; Numerical Methods; Machine Learning optimization

3.7. Equation Solving

Perhaps the most satisfying SymPy feature is equation solving.

Suppose: $x^2 - 5x + 6 = 0$

```
x = sp.Symbol('x')

eq = x**2 - 5*x + 6
print(sp.solve(eq, x))
```

[2, 3]

- **Multiple Variables**

$$\begin{cases} x + y = 5 \\ 2x + y = 8 \end{cases}$$

```
x, y = sp.symbols('x y')

solutions = sp.solve((x + y - 5,
                      2*x + y - 8),
                       (x, y))

print(solutions)
```

{x: 3, y: 2}

SymPy solves the equations symbolically and returns the exact solution.

Summary

SymPy works with mathematical expressions exactly rather than numerically. It specializes in:

- Algebraic manipulation
- Derivatives
- Integrals
- Partial derivatives
- Limits
- Taylor and Maclaurin series
- Equation solving

Think of SymPy as a computer algebra system that behaves more like a mathematician than a calculator.

Now, at this point you have grasped the common methods of NumPy, SciPy, SymPy, and their connections. Together, **NumPy stores the numbers, SciPy analyzes the numbers, and SymPy understands the mathematics behind the numbers.**

References

https://docs.sympy.org/latest/index.html

https://www.geeksforgeeks.org/data-science/sympy-tutorial/

https://www.datacamp.com/tutorial/sympy

Hand-On Exercises:

</> Ex 3. Python Exercises

Next Section:

</> 31. Python: Pandas Series